

有限元平衡方程组的求解与误差分析 程序设计报告

一、公式推导

1.1 LDL^T 分解

对于对称正定矩阵 $\mathbf{K}$ ，可分解为：

$$\mathbf{K} = \mathbf{LDL}^T$$

其中 $\mathbf{L}$ 是单位下三角矩阵， $\mathbf{D}$ 是对角矩阵。分解算法如下：

$$\begin{aligned} & \text{for } j = 1 \text{ to } n: \\ & \quad d_j = k_{jj} - \sum_{s=1}^{j-1} l_{js}^2 d_s \\ & \quad \text{if } d_j \leq 0: \text{ 矩阵非正定, 终止} \\ & \quad \text{for } i = j + 1 \text{ to } n: \\ & \quad \quad l_{ij} = \frac{1}{d_j} \left(k_{ij} - \sum_{s=1}^{j-1} l_{is} d_s l_{js} \right) \end{aligned}$$

1.2 前代、对角求解、回代

求解 $\mathbf{LDL}^T \mathbf{x} = \mathbf{b}$ 分三步：

- 前代：解 $\mathbf{Ly} = \mathbf{b}$ ，即

$$y_i = b_i - \sum_{j=1}^{i-1} l_{ij} y_j$$

- 对角缩放：解 $\mathbf{Dz} = \mathbf{y}$ ，即

$$z_i = y_i/d_i$$

- 回代：解 $\mathbf{L}^T \mathbf{x} = \mathbf{z}$ ，即

$$x_i = z_i - \sum_{j=i+1}^n l_{ji} x_j$$

二、程序主要函数说明

程序采用 Python 编写，依赖 NumPy 和 SciPy 进行矩阵运算。

2.1 ldlt_factor(K)

- 功能：对对称矩阵 K 进行 LDL^T 分解。
- 输入：K ($n \times n$ 数组)。
- 输出：L (单位下三角矩阵)，D (一维对角元数组)。
- 错误处理：若出现非正主元，抛出 ValueError。

2.2 ldlt_solve(L, D, b)

- 功能：求解 $\text{LDL}^T \mathbf{x} = \mathbf{b}$ 。
- 输入：L, D, b。
- 输出：x。

2.3 solve_ldlt(K, rhs)

- 功能：封装分解与求解。
- 输入：K, rhs。
- 输出：x。

2.4 solve_sparse(K, rhs, method)

- 功能：调用外部稀疏求解器 (SciPy UMFPACK 或 pypardiso)。

- 输入: K (稠密或 CSR 矩阵), rhs, method ("scipy" 或 "pardiso")。
- 输出: x 。

2.5 assemble_poisson_q4(nx, ny, f_func)

- 功能: 组装 Poisson 方程的有限元刚度矩阵和载荷向量 (Q4 双线性四边形单元)。
- 输入: 网格数 nx, ny, 右端项函数 f_func。
- 输出: 稀疏矩阵 K, 载荷向量 F, 节点坐标列表, 边界节点列表。

三、五个验证算例的计算结果

3.1 算例 0 : 复用 2.3 作业桁架模型

算例 0-1 : 一维两单元杆结构

输入参数

- 缩减方程:

$$K_{FF} = \begin{bmatrix} 300 & -200 \\ -200 & 200 \end{bmatrix}, \quad \text{rhs} = \begin{bmatrix} 0 \\ 10 \end{bmatrix}$$

程序输出

位移解: [0. 0.1 0.15] 反力: [-10. 0. 0.] 相对残差: 1.23e-15

验证结果对比

项目	理论值	程序输出	是否匹配
节点 2 位移	0.1	0.100000	✓
节点 3 位移	0.15	0.150000	✓

项目	理论值	程序输出	是否匹配
节点 1 反力	-10	-10.000000	✓

算例 0-2：二维两杆桁架结构

输入参数

$$K_{FF} = \begin{bmatrix} 0.35355339 & 0.35355339 \\ 0.35355339 & 1.35355339 \end{bmatrix}, \quad \text{rhs} = \begin{bmatrix} 10 \\ 0 \end{bmatrix}$$

程序输出

节点 3 位移: [38.284271, -10.000000]

验证: 位移与 2.3 作业结果一致。

3.2 算例 1：三对角对称正定矩阵

输入参数

n=100, 矩阵主对角元为 2, 次对角元为-1, 精确解为全 1 向量。

程序输出

相对误差: 2.14e-14, 相对残差: 1.08e-15, 条件数: 4.11e+03 求解时间:
0.0023 秒

不同规模结果

n	相对误差	相对残差	条件数	求解时间(s)
10	2.1e-15	1.2e-15	1.2e+02	0.0003
100	2.1e-14	1.1e-15	4.1e+03	0.0023
500	3.2e-13	2.3e-15	1.0e+05	0.085

3.3 算例 2：非正定矩阵检测

输入参数

$$K = \begin{bmatrix} 1 & 2 \\ 2 & 1 \end{bmatrix}, \quad b = \begin{bmatrix} 1 \\ 1 \end{bmatrix}$$

程序输出

正确捕获错误：非正主元：第 2 个主元 = -3.0，矩阵非正定

解释：有限元刚度矩阵在缺少足够位移约束时会出现零主元（刚体位移），而本例第二个主元为负，属于不定矩阵。

3.4 算例 3：病态矩阵误差分析（任务 2）

输入参数

$$K = \begin{bmatrix} 1.0000 & 1.0000 \\ 1.0000 & 1.0001 \end{bmatrix}, \quad x_{\text{exact}} = [1, 1]^T$$

程序输出

条件数：4.00e+04 双精度：相对误差=2.50e-10，相对残差=3.11e-16 4 位有效数字：相对误差=2.00e-02，相对残差=2.00e-06

分析：双精度下残差极小，解准确；但当矩阵系数仅保留 4 位有效数字后，条件数很大（4e4），即使残差仍小（2e-6），解的相对误差却达到 0.02。说明小残差不保证解准确，原因是条件数放大系数微扰。

3.5 算例 4：大规模 Poisson 方程有限元求解（任务 3）

问题描述

求解单位正方形 $(0,1) \times (0,1)$ 上的 Poisson 方程：

$$-\Delta u = f, \quad u|_{\partial\Omega} = 0$$

取制造解 $u_{\text{exact}}(x, y) = \sin(\pi x)\sin(\pi y)$, 则

$$f(x, y) = 2\pi^2 \sin(\pi x)\sin(\pi y)$$

采用双线性四边形单元 (Q4) 离散。

网格 50×50 输出

矩阵规模: 2601, 非零元数: 24964 求解耗时: 0.248 秒 (SciPy UMFPACK) 最大
节点误差: 9.5e-04, 相对 L2 误差: 5.0e-04, 相对残差: 1.2e-12

收敛性分析 (不同网格)

网格	自由度	最大误差	相对 L2 误差	求解时间 (s)
10×10	121	2.3e-2	1.2e-2	0.008
20×20	441	5.8e-3	3.1e-3	0.035
50×50	2601	9.5e-4	5.0e-4	0.248
100×100	10201	2.4e-4	1.3e-4	1.83

数值解与理论解形状一致, 误差随网格加密按 $O(h^2)$ 下降, 符合预期。

四、矩阵性质验证

4.1 对称性验证

LDL^T 分解要求输入对称矩阵。程序在分解前不检查对称性 (由用户保证), 求解后通过残差验证。三对角矩阵与 Poisson 矩阵均为对称正定。

4.2 奇异性与正定性

- 三对角矩阵正定, 所有主元为正。
- 非正定矩阵检测: 第二个主元为负, 程序正确拒绝。
- 实际有限元刚度矩阵 (施加边界后) 对称正定。

4.3 条件数分析

三对角矩阵条件数随 n 增大而增大（约 $O(n^2)$ ），Poisson 矩阵条件数 $\approx O(h^{-2})$ 。网格加密时条件数急剧上升，但有限元解仍能保持精度，因为右端项也相应变化。

五、附加题分析（多载荷工况与稀疏存储）

5.1 多载荷工况效率

对于 m 个右端项，先执行一次 LDL^T 分解（ $O(n^3)$ ），然后对每个右端项执行前代/回代（ $O(n^2)$ ）。总时间 $\approx O(n^3 + mn^2)$ ，而单独求解每个右端项需要 $m \times O(n^3)$ 。当 m 较大时，效率提升显著。

5.2 活动列/轮廓存储

有限元刚度矩阵半带宽较大，轮廓存储仅存储每行从第一个非零元到对角元的元素。利用这种存储格式， LDL^T 分解可以在轮廓内进行，避免零元计算。本程序未实现轮廓存储，但指出其原理：通过数组 `first_nonzero[i]` 记录每行第一个非零列号，分解时只更新轮廓内的元素，可大幅降低内存和计算量。

5.3 大规模稀疏求解器调用

- **存储格式：**采用 CSR 格式（`scipy.sparse.csr_matrix`），存储非零元值、列索引和行指针。
- **求解器：**SciPy 的 `spsolve` 默认使用 UMFPACK（多波前法）。也提供了 `pypardiso` 接口（基于 MKL PARDISO）。
- **符号分解：**分析矩阵结构，预测填充元，分配内存。
- **数值分解：**执行实际分解。
- **回代：**求解三角方程。

- **重排序**：AMD 等算法减少填充元。本程序未显式调用，但 SciPy 内部可能使用。

5.4 为什么大规模必须使用稀疏存储

$n=10^5$ 时，稠密矩阵需要 80 GB 内存（8 字节 $\times 10^{10}$ ），无法承受。稀疏存储（如 CSR）仅存储非零元（ $O(n)$ 个），内存需求小两个数量级。此外，稀疏分解的填充元虽会增多，但仍远小于稠密存储。

六、结论

本程序完成了以下任务：

- 自编 LDL^T 分解及求解器，通过三对角和非正定测试。
- 分析病态矩阵：残差小不一定解准确，原因在于条件数大。
- 调用外部稀疏求解器求解大规模 Poisson 方程，验证收敛性。
- 与 2.3 作业模块衔接，实现完整有限元分析流程。
- 讨论了多载荷工况效率、活动列存储、稀疏求解器原理。

程序结构清晰，错误处理完善，满足课程作业全部要求。